No of additional sheets 6

LOURDES MATHA COLLEGE OF SCIENCE & TECHNOLOGY

LOURDES HILLS, KUTTICHAL P. O., THIRUVANANTHAPURAMM - 695 574

-30/30

ANSWER BOOK

Name DEEPAK.V Class No. 17 Branch MCA (2021-23)

Subject Design and Analysis of Algorithms Semester S3

Mark Tabulation

		Q. 1	Q. 2	Q. 3	Q. 4	Q. 5	Q. 6	Q. 7	Q. 8	Q. 9	Q. 10	Q. 11	Q. 12	Q. 13	Q. 14	Q. 15
Mark Divisions																
Total																
Grand Total																

PART-B

1. N Queens

→ N Queens problem is a game like ~~problem~~ problem in which we want to place N queens in such a way that no queens can prue each other.

→ Suppose consider a matrix of size, say n = 4 i.e, in 4x4 matrix. The 4 queens need to place in such a way that they can't prue, cut or even attack each other.

→ The queens are placed in a corresponding rows. i.e, queen 1 (Q_1) is placed in 1st row, queen 2 (Q_2) is placed in 2nd row, etc..

finally the nth Queen is placed at nth row itself.

→ The position of column only gets changed which doing N Queens problem and the output of the N Queens problem is the set of columns positions of each Queens in an order.

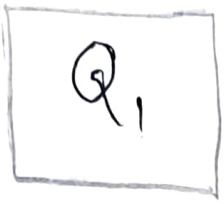
→ This is the general idea of the n queens problem. Let's analyze how to solve such problems.

→ This kind of problem is already understood that it's game like problem and remember game like problems and Artificial Intelligence (AI) problems are commonly solved using "Back Tracking".

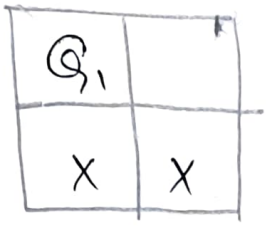
→ The Back Tracking is an algorithm approach to solve optimization problem. In which we focus on one solution path until it's impossible or not legitimate to move further. If a problems like mentioned occur during propagation of the problem. Back Tracking chooses another branch of solution path.

→ Back Tracking propagates using state space tree.

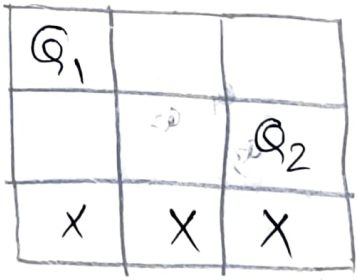
Suppose Consider the N=1, N=2, and N=3 for 3 set of problems and lets try to find out solutions corresponding for it



N=1 q can hold only 1 queen no problem.



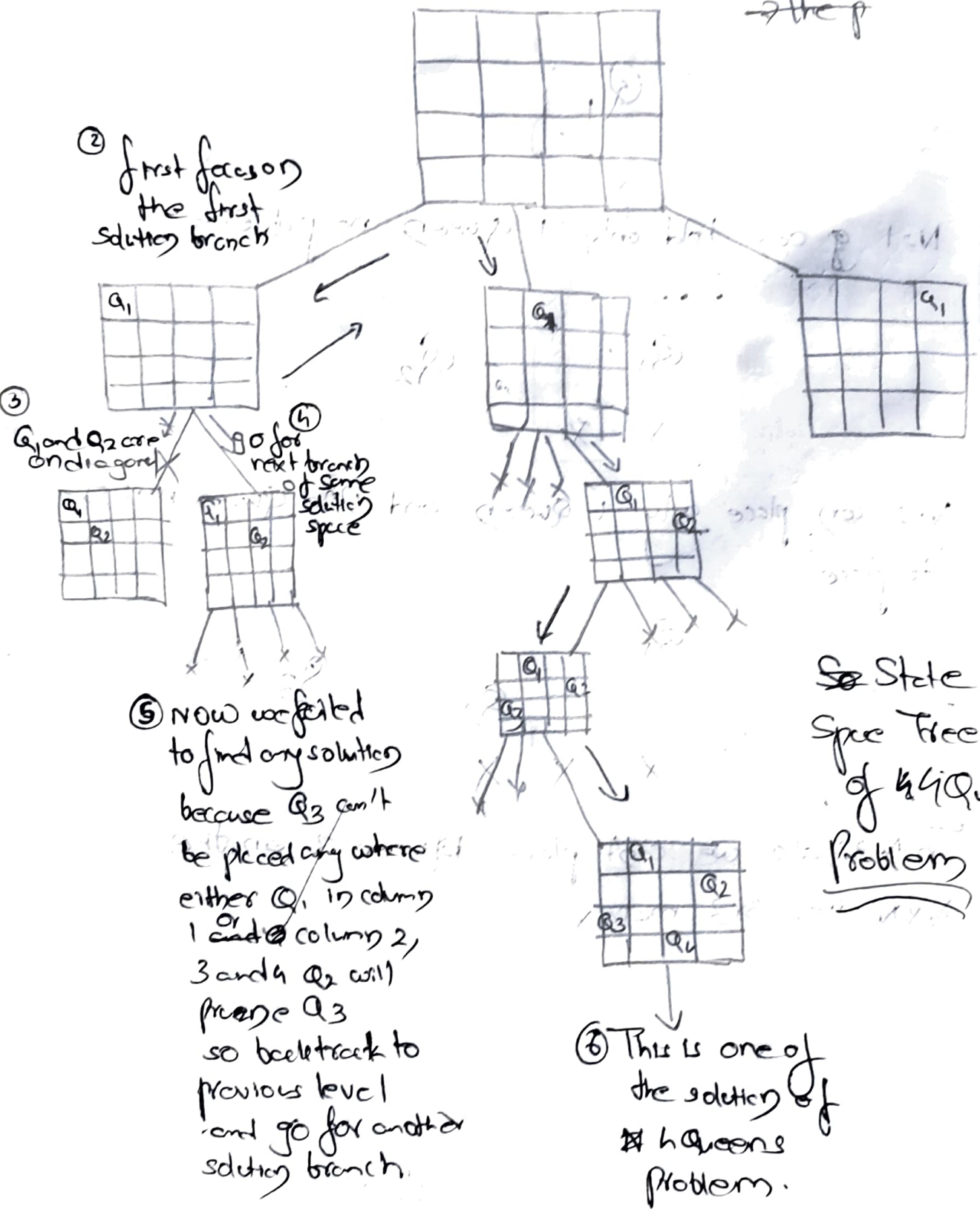
N=2 can place only 1 queen and 2nd queen misses to place



In N=3 also we can't place Queen 3 on the NxN i.e, 3x3 matrix

Now, Let's try on N=4. i.e. NxN = 4x4 matrix
we need to place 4 Queens Q_1, Q_2, Q_3, Q_4

Step 1:- Initialize the condition stated above,



State Space Tree of 4 Queens Problem

From the solution space tree we can observe some facts, that,

→ If there is one point (i,j) and (k,l) for two queens respectively then.

→ The Queens to be diagonal either.

first solution, $(j-l) = (i-k)$, OR

second solution, $(j-l) = (k-i)$

→ So we can conclude that, if two queens gets digonal.

$$|j-l| = |i-k|$$

Algorithm N-Queens

```
place (k,i)
{
  for j → 1 to k-1
    do if (x[j] = i)
    or if (abs(x[j] - l) == abs(i-k))
      return FALSE

  return TRUE
}
```

```
NQueens(k, n)
{
  for i ← 1 to n
    do if place(k, i) then
    {
      x[k] ← i
      if (k == n) then
        write (x[1 ... n])
      else
        N-Queens(k+1, n);
    }
}
```

Here, x[] is a global array which holds the solution. k is the Queen and i is the row and j is the column to be placed

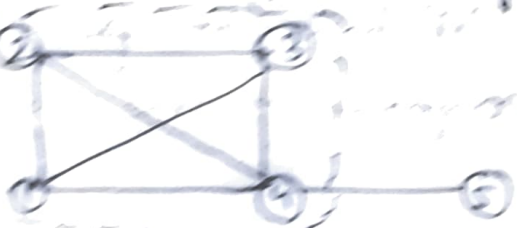
9. Clique Problem

→ The Clique problem is to simply to find clique in a the graph G(V, E).

→ The clique a subgraph of a graph in which all the the vertices in the g subgraph connects with each other. i.e, a complete graph need to obtain from a subgraph.

→ A simple terms maximum ~~K~~ vertex x k clique need is be obtained from Graph G. Corresponding optimize problem.

eg:



Here, the Clique is 4

→ The Clique problem ~~is~~ is NP Complete if and only if the Clique problem is #NP as well as NP hard.

→ Clique Problem is NP

→ The Clique Problem is NP then the clique problem may not solve in polynomial time. But it can be verified in polynomial time.

→ ~~The Clq~~ for verifying a graph contain a clique of k size we need a Certificate to verify.

→ The certificate here is the set of nodes 'S' ~~consists consisting of~~ which is on clique and $S \subseteq G(V)$

→ for checking the ~~certifite~~ certificate, only $O(n^2)$ time is needed. So, we can conclude that:

→ Clique Problem is NP.

Clique Problem is NP-Hard

→ for a Problem to be NP-hard The ~~pro~~ a problem can be reduced ~~to~~ in a polynomial ~~to~~ time to a corresponding problem.

→ for ~~first~~ proving Clique is a NP-Hard problem we need a another NP-problem which already proven i.e, Boolean Satisfyability Problem (SAT)

→ Our aim to reduce SAT problem ~~to~~ in order to solve clique problem ~~in~~ polynomial time.

→ Suppose consider a boolean expression

$(x_1 \wedge x_2) \vee (x_2' \wedge x_3) \vee (x_1' \wedge x_3')$

where, $\wedge$ is "and" operator

$\vee$ is "or" operator

x' is complement

→ we need to construct a graph from this boolean expression in such a way that

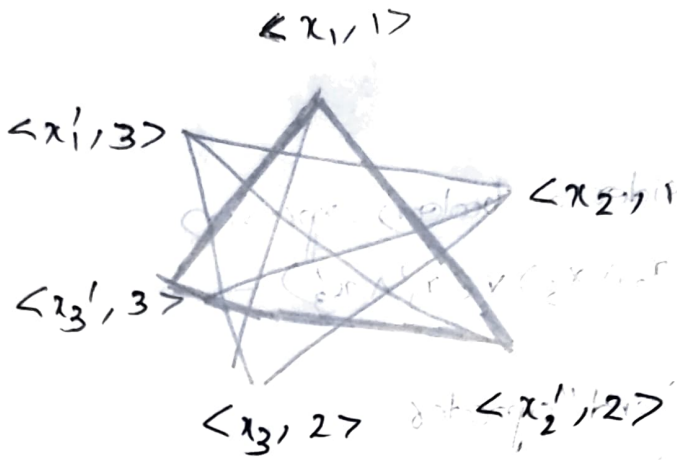
→ no same clause elements gets an edge in between.

→ element with it's corresponding complement doesn't form an edge in between.

for that first write down the boolean expression in <element, clause> format

i.e, $\langle x_1, 1\rangle$; $\langle x_2, 1\rangle$; $\langle x_2', 2\rangle$; $\langle x_3, 2\rangle$; $\langle x_1', 3\rangle$; $\langle x_3', 3\rangle$

Then construct graph.



→ Here, there are more than one solutions or cliques but the maximum clique we can obtain here is k=3.

→ So we can represent it as (x_1, x_2', x_3') or (1,0,0) as one of the solution.

→ So since we can reduce the solution from SAT to Clique problem and it is reduced in polynomial time.

→ We can conclude the Clique problem is NP-hard

→ Since, Clique problem is both NP-hard and NP ~~complete~~ the Clique problem is NP-Complete

PART-A

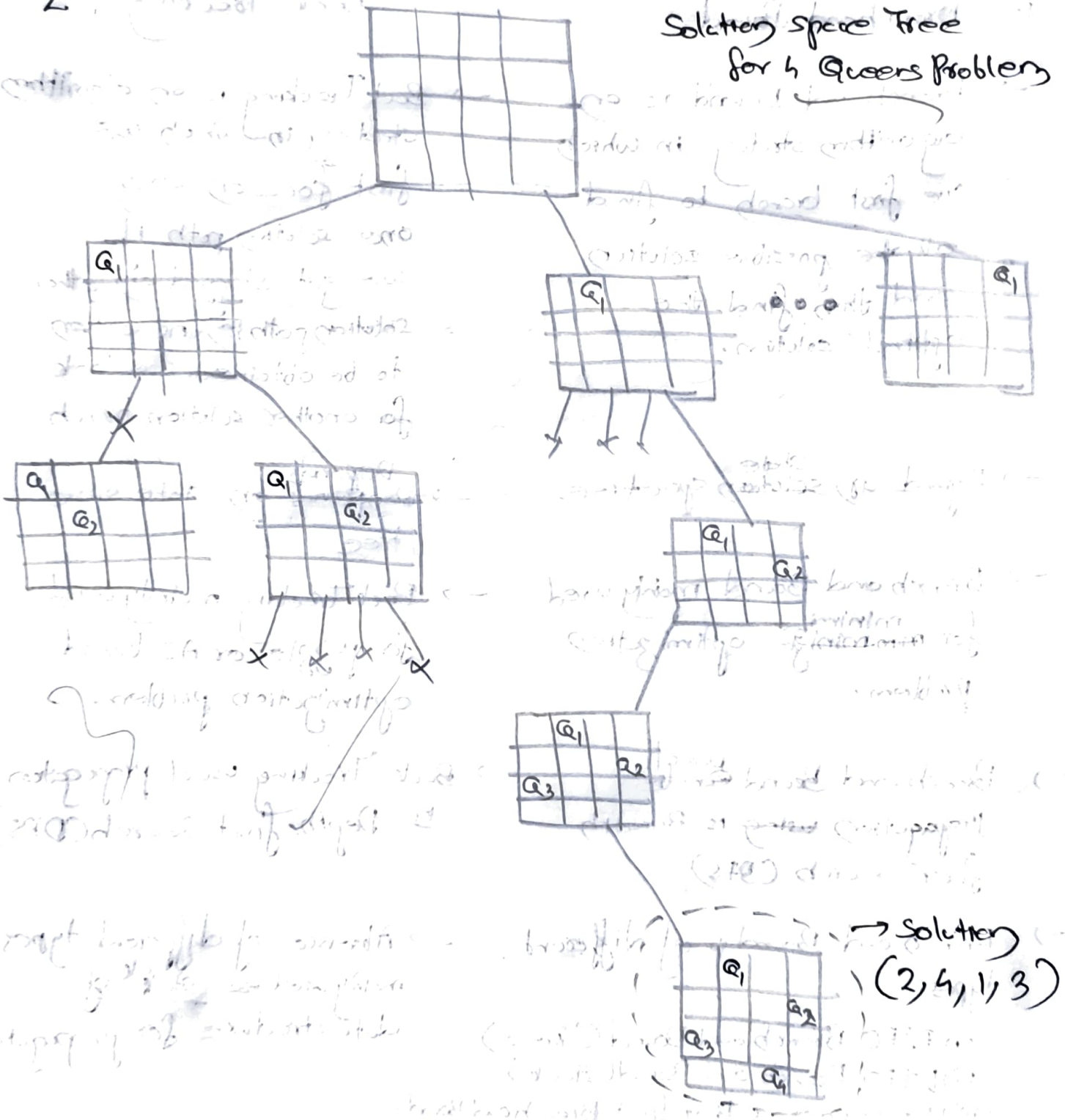
1. Branch and Bound	Back Tracking
→ Branch and Bound is an algorithm strategy in which we first branch to find all the possible solution and then find the optimal solution.	→ Back Tracking is an algorithm strategy in which we first focus on only one solution path if we get stopped on that solution path i.e, no solution to be obtained we look for another solution branch
→ Depends on state space tree	→ Depends on state space tree
→ Branch and Bound mainly used for minimize optimization problem.	→ Back Tracking mainly used for puzzle or AI based optimization problem
→ Branch and Bound usual propagation is Breadth first search (BFS)	→ Back Tracking usual propagation is Depth first search (DFS)
→ Branch and Bound is of different types. (i) FIFO Branch and Bound (Queue) (ii) LIFO Branch and Bound (Stack) (iii) Least cost or Best first Branch and Bound.	→ Absence of different types mainly used stack of data structure for propagation

Branchand Band
→ Some examples
N^2-1 puzzle
~~8 Queens~~, Job-Schedule,
Knapsack with Branchand
Bag, etc...

Back Tracking
→ Some examples are
~~4 2 N~~ Queens ~~puzzle~~ problem,
Min-max problem,
etc...

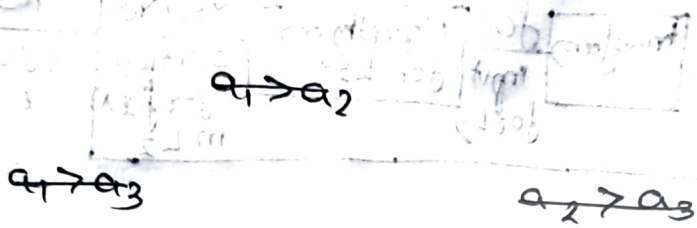
2.

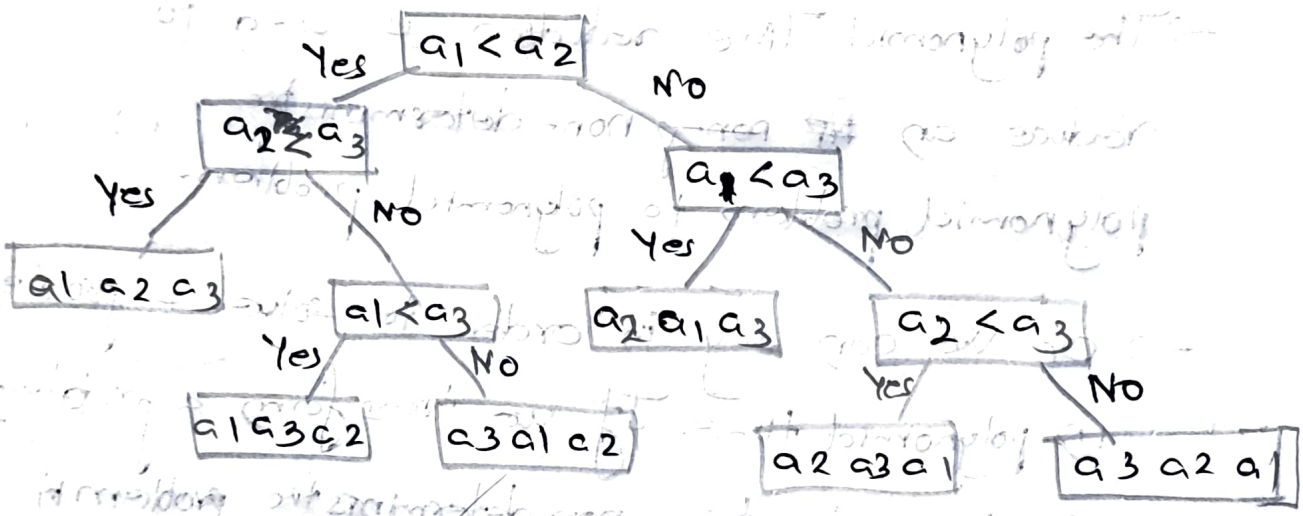
Solution space Tree
for 4 Queens Problem



→ Solution
(2, 4, 1, 3)

3. → Suppose consider the problem of sorting 3 numbers a_1, a_2, a_3 then. The decision tree becomes.

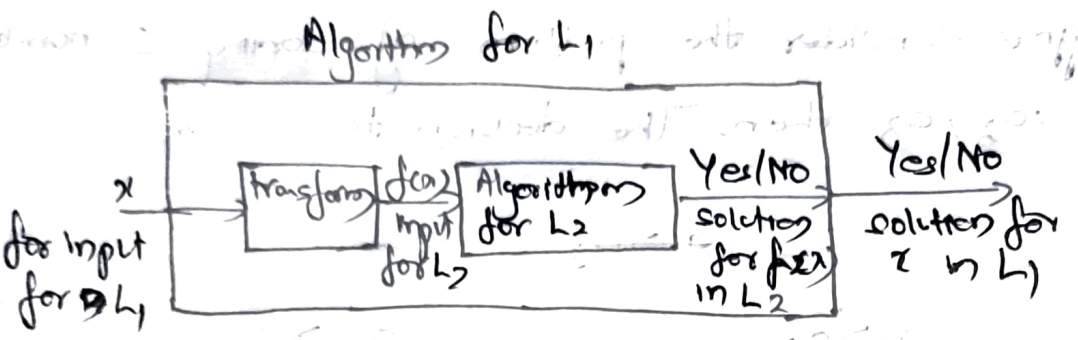




→ The main purpose of decision tree for comparison based sorting is to find the lower bound time complexity.

→ From decision tree the smallest maximum height of decision tree is the lower bound time complexity

4. Polynomial Time Reduction



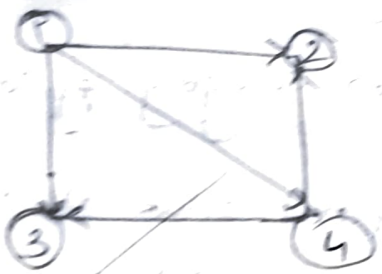
→ The polynomial Time reduction is used to reduce an ~~NP~~ non-deterministic polynomial problem to polynomial problem

→ Or we can say in order to solve a problem in polynomial time. If we transform a problem ~~A to B by trans~~ non deterministic problem A by transforming the input to A and to polynomial time problem B and find the solution for the Algorithm ~~A~~ A ~~to~~ from Algorithm B in polynomial time then, we can say ~~that polynom~~ Algorithm A is ~~reduce~~ reduced to polynomial time

5. Vertex Cover problem

→ The Vertex Cover problem is a type of problem from a graph, the ~~word~~ ~~no. of word~~ vertices which is covering the maximum cover over all other vertices in a graph G.

→ The o/p of the Vertex cover is a set of vertices which can cover all other vertices of a graph G.



Here in this example
The vertex cover can be {1,3}
~~{4,3} / {1,3} or both~~

Here the edges are
{(1,2), (1,4), (4,3), (1,3)}
the ~~word~~ vertex 1 can cover all its other vertices

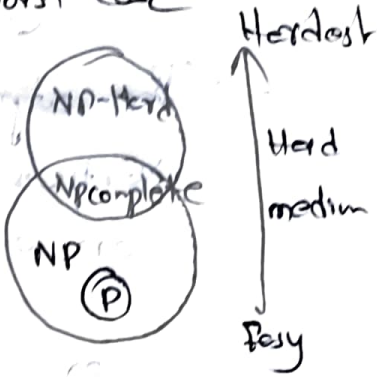
6. P-Class

-> The P-class are the one of the category of complexity classes in which

-> A algorithm problem can be solved in Polynomial time.

ie, if a algorithm can compute in polynomial time $P(n)$ then it's worst case time complexity is $O(p(n))$.

eg:- Quick sort, Merge Sort etc..



NP-Class

-> The NP-Class are the another category of complexity of classes in which.

-> The Algorithm problem may not solve it in polynomial time but can be solved in verified in polynomial time.

eg:- Vertex cover, Clique problem etc. ..

-> NP-class also known as Non-deterministic polynomial.